

Rail Nova: A Full-Stack Indian Railway Train Booking System with Real-Time Seat and Tatkal Queue Management

Nigam Mehta

mehtanigam3024@gmail.com

Pursuing B.Tech in Electronics Engineering (VLSI Design and Technology)

Abstract—Rail Nova is a full-stack web-based Indian railway reservation system designed to provide an integrated workflow for train discovery, passenger management, seat reservation, Tatkal booking, payment processing, PNR tracking, and booking cancellation. The system combines a Vanilla HTML/JavaScript frontend with a FastAPI backend, SQLAlchemy ORM, SQLite database, and WebSocket-based real-time communication. The implementation models practical railway-booking concepts such as confirmed (CNF), reservation-against-cancellation (RAC), and waitlist (WL) states, temporary seat locks during payment, class-dependent Tatkal opening times, a live Tatkal queue, and automatic cleanup of expired temporary resources. JWT authentication and password hashing are used to protect user accounts, while request logging and login rate limiting provide additional operational safeguards. This report presents the system architecture, database model, booking workflow, algorithms, APIs, security mechanisms, testing approach, implementation results, limitations, and future enhancements. Screenshots from the implemented application are included to demonstrate the major user-interface stages.

Index Terms—Railway reservation, train booking, FastAPI, SQLite, WebSocket, Tatkal, JWT, seat allocation, eWallet, full-stack web application.

I. INTRODUCTION

Railway reservation systems must coordinate large amounts of train, station, passenger, seat, payment, and schedule information while providing a responsive user experience. A booking request is not simply a database insertion: the system must validate the route, identify an eligible train and coach class, determine availability, temporarily protect seats during payment, assign a reservation status, and maintain a consistent booking record.

Rail Nova was developed as a practical implementation of these requirements. It provides a single-page booking interface backed by an asynchronous FastAPI service. The application supports train search, user authentication, passenger details, seat allocation, Tatkal booking, PNR lookup, cancellation, eWallet transactions, and real-time updates.

The project also demonstrates how conventional relational database operations can be combined with asynchronous WebSocket communication. This is particularly useful for railway scenarios where multiple users may observe or compete for the same limited seat inventory.

II. PROBLEM STATEMENT

A simplified booking application can store a train and passenger record, but a realistic reservation workflow requires several additional mechanisms:

- search over a large station and train dataset;
- authenticated user sessions;
- class-specific seat inventory;
- temporary locking of seats while payment is in progress;
- CNF/RAC/WL allocation logic;
- Tatkal time-window validation and queue management;
- payment confirmation and wallet transactions;
- PNR-based booking retrieval and cancellation;
- real-time availability and queue updates; and
- cleanup of expired temporary resources.

Rail Nova addresses these requirements through a modular backend containing routers, services, database models, schemas, middleware, and utility modules.

III. OBJECTIVES

The main objectives of the project are:

- 1) To develop an end-to-end railway ticket booking application.
- 2) To implement train and station search using structured railway datasets.
- 3) To implement authenticated registration and login using JWT.
- 4) To manage seat availability for different coach classes.
- 5) To implement CNF, RAC, and WL booking states.
- 6) To implement Tatkal time-gated booking and a live queue.
- 7) To provide mock online payment and eWallet payment flows.
- 8) To support PNR lookup and cancellation with wallet refund.
- 9) To provide WebSocket-based real-time updates.
- 10) To maintain a clean and modular software architecture suitable for future database and payment upgrades.

IV. SYSTEM OVERVIEW

Rail Nova consists of three principal layers:

- 1) **Presentation layer:** Vanilla HTML and JavaScript with Tailwind CSS for the browser interface.

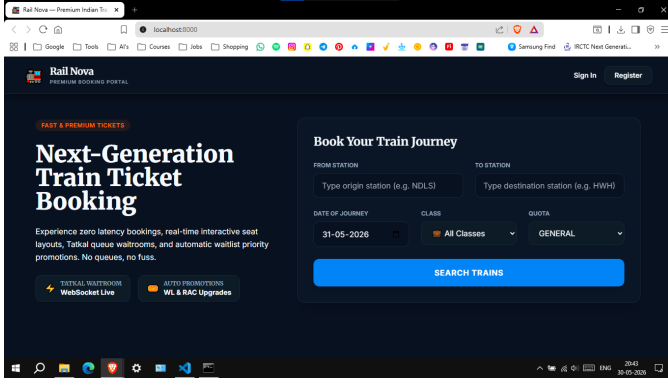


Fig. 1. Rail Nova home page and train-search interface.

TABLE I
TECHNOLOGY STACK

Component	Technology / Purpose
Frontend	HTML, Vanilla JavaScript, Tailwind CSS
Backend	Python, FastAPI, Uvicorn
ORM	SQLAlchemy
Database	SQLite with asynchronous SQLite support
Validation	Pydantic v2 / pydantic-settings
Authentication	JWT with password hashing
Real-time layer	WebSockets
Dataset	Indian railway stations, trains and stoppages in CSV format
Payment	Mock Razorpay-style order/verification and eWallet
Testing	Python API integration test script

- 2) **Application layer:** FastAPI routers, service classes, authentication dependencies, middleware, and WebSocket endpoints.
- 3) **Data layer:** SQLite accessed through SQLAlchemy, together with CSV datasets containing railway station, train, and stoppage information.

The frontend communicates with the backend using REST-style HTTP requests. Long-lived WebSocket connections are used when a user needs live seat availability or Tatkal queue status.

Figure 1 shows the implemented home page. The interface provides source and destination fields, journey date, coach class, quota selection, and a train-search action.

V. TECHNOLOGY STACK

VI. SYSTEM ARCHITECTURE

The high-level architecture is organized as shown conceptually below:

- 1) The browser maintains application state such as the authentication token, selected train, selected class, booking details, and active WebSocket connections.
- 2) API requests are routed through FastAPI endpoints.
- 3) Authentication dependencies validate JWT bearer tokens for protected operations.
- 4) Routers delegate business operations to service classes.

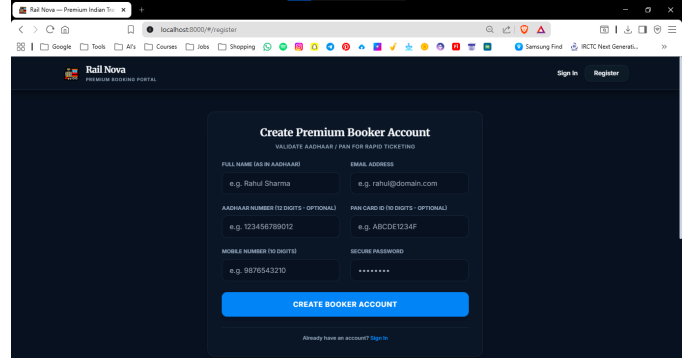


Fig. 2. User registration interface.

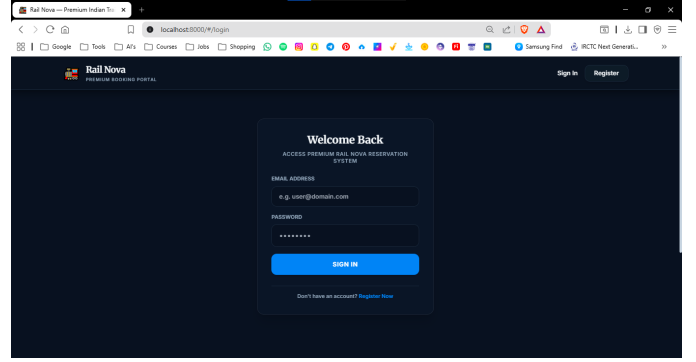


Fig. 3. User login interface.

- 5) Service classes perform validation and database transactions using SQLAlchemy.
- 6) WebSocket endpoints maintain active connections and push availability or Tatkal status updates.
- 7) Background tasks periodically remove expired seat locks and manage Tatkal queue state.

The separation between routers and services reduces coupling. For example, the booking router exposes the API contract while the booking service contains the booking workflow.

VII. FUNCTIONAL MODULES

A. User Authentication

The registration page collects user information and creates an account. The login operation returns a JWT access token, which the frontend stores locally and attaches to subsequent protected API calls.

A simplified authentication sequence is:

Credentials → Verification → JWT → Bearer Token. (1)

The backend provides endpoints for registration, login, logout, current-user retrieval, profile updates, and password changes. Login attempts are rate-limited to reduce repeated authentication abuse.

B. Train and Station Search

Rail Nova uses station and train master data together with train stoppage records. Station autocomplete can query the

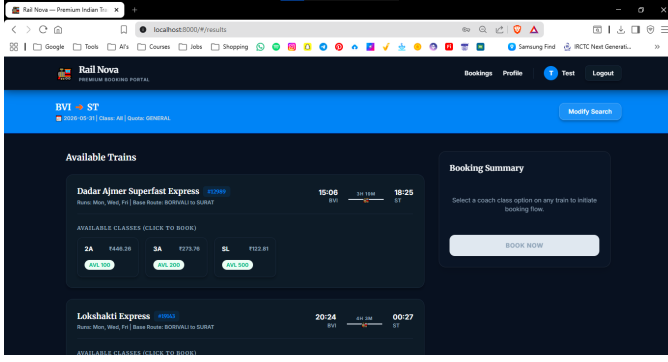


Fig. 4. Train-search results with coach-class availability.

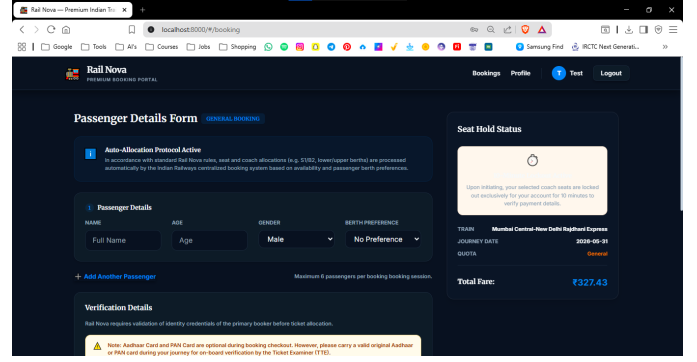


Fig. 5. Passenger details and booking stage.

station dataset, while train search accepts source, destination, date, and class parameters.

The system also considers intermediate stoppages when determining a journey segment. If two selected stations are valid stops on the same train and occur in the correct order, the system can use the segment rather than blindly treating the complete train route as the journey.

C. Seat Availability and Allocation

Seat records maintain values including total seats, confirmed count, RAC count, waitlist count, and temporarily locked seats. Availability can therefore be represented by

$$A = T - C - L, \quad (2)$$

where T is total inventory, C is confirmed allocation, and L represents currently locked seats. RAC and WL values are tracked separately because they represent different reservation states.

The implementation maps coach classes to coach prefixes and class-dependent capacities. For example, AC First Class is modeled with a lower capacity than common 72-seat classes, while AC Chair Car is represented separately.

D. Booking and Temporary Seat Locks

A booking starts by validating the selected train and coach class. The system obtains or creates the relevant seat-availability record and considers active locks. During the payment stage, temporary seat locks protect inventory from simultaneous competing bookings.

A lock can be viewed as a reservation with an expiry time:

$$t_{\text{expire}} = t_{\text{lock}} + \Delta t, \quad (3)$$

where Δt is the configured lock duration. The project uses a ten-minute temporary seat-lock concept. Expired locks are removed by background cleanup processing.

This design prevents a user from holding seats indefinitely when payment is abandoned.

E. Tatkal Booking

Tatkal booking is implemented as a time-gated operation. The system identifies whether the selected coach class is AC or non-AC and applies the corresponding opening time one day before the journey date:

$$t_{\text{open}} = \begin{cases} 10:00, & \text{AC class} \\ 11:00, & \text{Non-AC class.} \end{cases} \quad (4)$$

Users can join a Tatkal queue after the window opens. The first 50 queue positions are treated as active windows, while subsequent users enter a waiting state. Queue status is persisted in the database and exposed through an API and WebSocket connection.

F. Real-Time WebSocket Communication

Two application-specific WebSocket mechanisms are implemented.

Seat availability channel:

```
ws://localhost:8000/ws/seats/{train_number}/{
  journey_date}
```

On connection, the backend sends initial availability for supported classes. Booking events can trigger availability update signals to connected clients.

Tatkal queue channel:

```
ws://localhost:8000/ws/tatkal/{train_number}/{
  journey_date}/{coach_class}
```

The Tatkal WebSocket retrieves the latest queue status periodically, allowing the browser to reflect changes without requiring a manual page refresh.

G. Payment and eWallet

The payment workflow supports a mock Razorpay-style create-order and verification sequence. This is intentionally a simulation rather than a live payment gateway integration.

An additional eWallet mechanism maintains a balance and transaction history. The wallet supports top-up and payment operations. When a booking is cancelled, the project supports automatic refund to the eWallet.

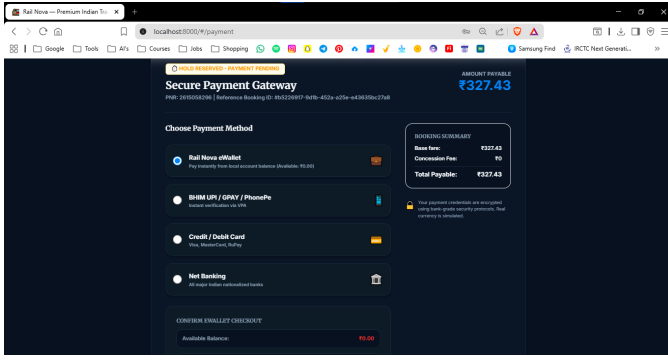


Fig. 6. Rail Nova payment interface.

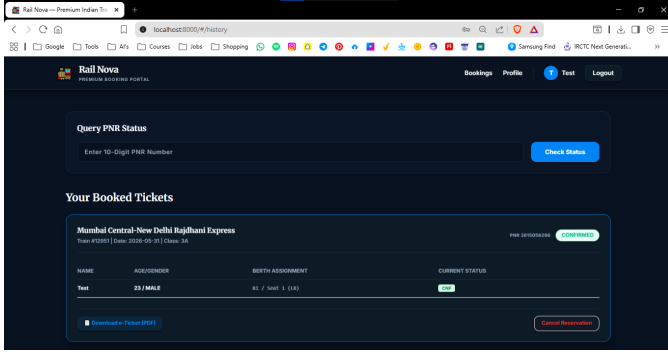


Fig. 7. Booked-ticket history and PNR information.

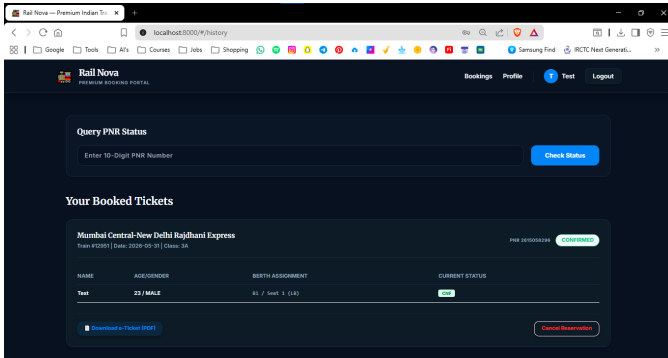


Fig. 8. PNR status lookup interface.

H. PNR Lookup and Cancellation

Each booking is associated with a generated PNR. The history interface allows a user to retrieve booking information and inspect reservation status. Cancellation changes the booking state and initiates the corresponding refund workflow.

VIII. DATABASE DESIGN

The database uses a relational model implemented with SQLite and SQLAlchemy ORM. The major tables are summarized in Table II.

The schema separates train master data from date-specific seat inventory. This is important because the same train may have different availability on different journey dates.

TABLE II
MAJOR DATABASE TABLES

Table	Role
users	Registered users and account information
trains	Train master records and coach availability flags
stations	Station names, codes, coordinates, zones and states
train_stoppages	Intermediate stops for each train
bookings	Booking records, PNR and reservation status
passengers	Passengers associated with bookings
seats	Dynamic train/date/class seat inventory
seat_locks	Temporary seat reservations during payment
payments	Mock payment records and verification state
tatkal_queue	Tatkal queue positions and state
wallet_transactions	eWallet credits and debits

TABLE III
REPRESENTATIVE API ENDPOINTS

Endpoint	Function
POST /api/auth/register	Register user
POST /api/auth/login	Authenticate user and return JWT
GET /api/auth/me	Retrieve authenticated profile
GET /api/trains/search	Search trains
GET /api/trains/{number}/availability	Retrieve class availability
POST /api/bookings	Create booking
GET /api/bookings/{pnr}	Retrieve booking by PNR
DELETE /api/bookings/{pnr}	Cancel booking
POST /api/bookings/tatkal/join	Join Tatkal queue
GET /api/bookings/tatkal/status	Retrieve Tatkal status
POST /api/payments/create-order	Create mock payment order
POST /api/payments/verify	Verify mock payment

IX. REST API DESIGN

The API is organized by resource.

FastAPI also provides interactive API documentation through Swagger UI and ReDoc, which is useful during development and testing.

X. SECURITY AND RELIABILITY

Several mechanisms are incorporated into the implementation.

A. JWT Authentication

Protected API requests use bearer authentication. The frontend adds the token to the HTTP Authorization header. If the server returns an unauthorized response, the frontend clears the session and redirects the user to the unauthenticated state.

B. Password Protection

Passwords are not intended to be stored as plain text. The project uses password hashing utilities and JWT-based session authentication.

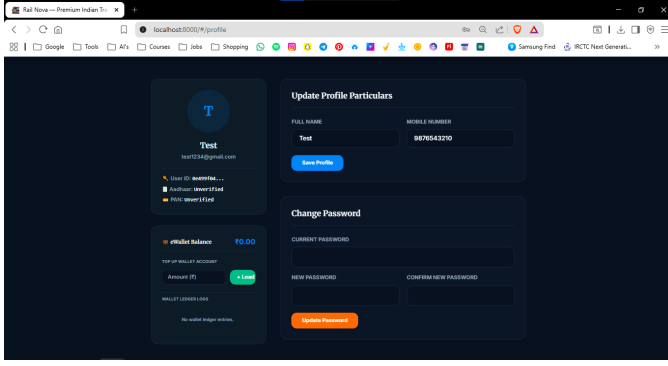


Fig. 9. User profile and account-management interface.

C. Login Rate Limiting

The login route uses an in-memory rate limiter allowing five login attempts per minute per IP address. This reduces repeated automated login attempts during a running server instance.

D. Request Logging

Middleware records request method, path, status, and latency. Such information can help identify slow endpoints and abnormal request patterns.

E. Automatic Cleanup

Background processing runs periodically to expire temporary seat locks and rotate Tatkal queue states. This prevents abandoned resources from remaining active indefinitely.

XI. FRONTEND DESIGN

The frontend is implemented as a single-page application without a heavy JavaScript framework. A central state object stores:

- authentication token and current user;
- station and train search parameters;
- search results;
- selected train and coach class;
- active booking information;
- Tatkal queue status;
- booking history and PNR result; and
- active WebSocket connections.

Navigation is controlled through URL hash routes such as home, login, register, results, booking, payment, history, and profile. This provides a lightweight client-side routing mechanism.

XII. BOOKING WORKFLOW

The normal booking flow can be summarized as:

- 1) User opens the home page.
- 2) Source, destination, date, and class are selected.
- 3) Backend searches for matching trains.
- 4) User selects a train and class.
- 5) Passenger details are submitted.
- 6) Availability and active locks are evaluated.
- 7) A temporary seat lock is created for the payment stage.

TABLE IV
REPRESENTATIVE TEST CASES

Test	Expected Result	Observed/Designed Behavior
User registration	Account created	Supported
User login	JWT returned	Supported
Invalid login	Request rejected	Supported
Train search	Matching trains shown	Supported
Class availability	Inventory returned	Supported
Booking	PNR generated	Supported
PNR lookup	Booking retrieved	Supported
Cancellation	Booking cancelled/refund initiated	Supported
Tatkal before opening	Request rejected	Supported
Tatkal queue	Position/status returned	Supported
Payment verification	Booking confirmation path completed	Supported as mock payment
WebSocket connection	Live status available	Supported

- 8) Payment is created and verified.
- 9) The booking is committed and a PNR is generated.
- 10) Availability update signals are broadcast to connected clients.

The workflow ensures that payment and inventory handling are coupled rather than treating payment as an unrelated operation.

XIII. TATKAL QUEUE WORKFLOW

The Tatkal workflow differs from normal booking:

- 1) User selects Tatkal quota and train/class.
- 2) Backend verifies the one-day-before opening rule.
- 3) If the window is closed, the request is rejected with the next opening time.
- 4) If open, the user can join the train/date/class queue.
- 5) Queue position is calculated from existing active/waiting entries.
- 6) Positions up to 50 receive an active ten-minute window; later users remain waiting.
- 7) The WebSocket periodically delivers the latest status.

This mechanism demonstrates the handling of a high-contention reservation feature without requiring a page refresh for every status change.

XIV. TESTING AND RESULTS

Testing focuses on the complete user workflow and API behavior. The project contains an integration test script for exercising the backend while the server is running. Representative test cases are listed in Table IV.

The screenshots in Figs. 4–8 demonstrate the principal implemented stages: train search, passenger entry, payment, booking history, PNR status, and profile management.

XV. IMPLEMENTATION CONSIDERATIONS

The project uses SQLite because it is zero-configuration and convenient for academic development. SQLAlchemy abstracts database operations and makes future migration to a larger relational database practical.

The application currently runs on a local development and test server (localhost) using Uvicorn. This setup is used for functional testing and academic demonstrations rather than public production deployment. The backend can serve the static frontend directly, allowing the project to run without a separate frontend development server.

The dataset includes approximately 7,000 Indian railway stations and a large train-stoppage CSV. The stoppage dataset is comparatively large, so initial database seeding can take additional time.

XVI. LIMITATIONS

The current academic implementation has several known limitations:

- Payment processing is mocked and is not connected to a real banking or payment provider.
- SQLite is appropriate for development but is not ideal for high-concurrency production deployment.
- Email-based account verification is not implemented.
- Login rate limiting is in memory and therefore resets when the server restarts.
- The auto-shutdown feature is designed for local/demo usage and should be disabled or replaced in production.
- Railway data is supplied through local datasets; production deployment would require an authoritative and continuously updated railway data source.

XVII. FUTURE SCOPE

The project can be extended in several directions:

- 1) Replace SQLite with PostgreSQL for concurrent production workloads.
- 2) Integrate a real payment gateway with secure server-side verification and webhooks.
- 3) Add email/SMS notifications for booking confirmation and cancellation.
- 4) Introduce Redis for distributed caching, rate limiting, and WebSocket coordination.
- 5) Add administrator dashboards for train, station, booking, and user management.
- 6) Implement stronger distributed locking and transactional inventory control for large-scale concurrent bookings.
- 7) Add accessibility improvements and multilingual support.
- 8) Deploy the application using containers and a reverse proxy with HTTPS.
- 9) Add observability through metrics, structured logs, and tracing.

XVIII. CONCLUSION

Rail Nova demonstrates the design and implementation of a complete railway ticket-booking workflow using modern web technologies. The system combines a lightweight browser

frontend, modular FastAPI backend, relational data model, JWT authentication, temporary seat locking, CNF/RAC/WL reservation states, Tatkal queue management, mock payments, PNR operations, and WebSocket-based real-time updates.

The project is particularly useful as an academic demonstration because it connects software architecture with practical reservation-system constraints. Rather than implementing only a CRUD booking form, Rail Nova models contention, temporary resource ownership, time-dependent Tatkal rules, payment stages, and real-time client updates. The modular structure also provides a clear path toward future migration to production-grade databases, payment services, distributed coordination, and deployment infrastructure.

ACKNOWLEDGMENT

The author acknowledges the academic project environment and open-source software ecosystem that made the implementation of this full-stack demonstration possible.

REFERENCES

- [1] FastAPI Documentation, "FastAPI: Modern, Fast (high-performance) web framework for building APIs with Python," available at <https://fastapi.tiangolo.com/>.
- [2] SQLAlchemy Documentation, "The Python SQL Toolkit and Object Relational Mapper," available at <https://www.sqlalchemy.org/>.
- [3] SQLite Documentation, "SQLite: Small. Fast. Reliable. Choose Any Three," available at <https://www.sqlite.org/docs.html>.
- [4] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, 2011.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.
- [6] Pydantic Documentation, "Data validation using Python type hints," available at <https://docs.pydantic.dev/>.
- [7] Uvicorn Documentation, "An ASGI web server implementation for Python," available at <https://www.uvicorn.org/>.